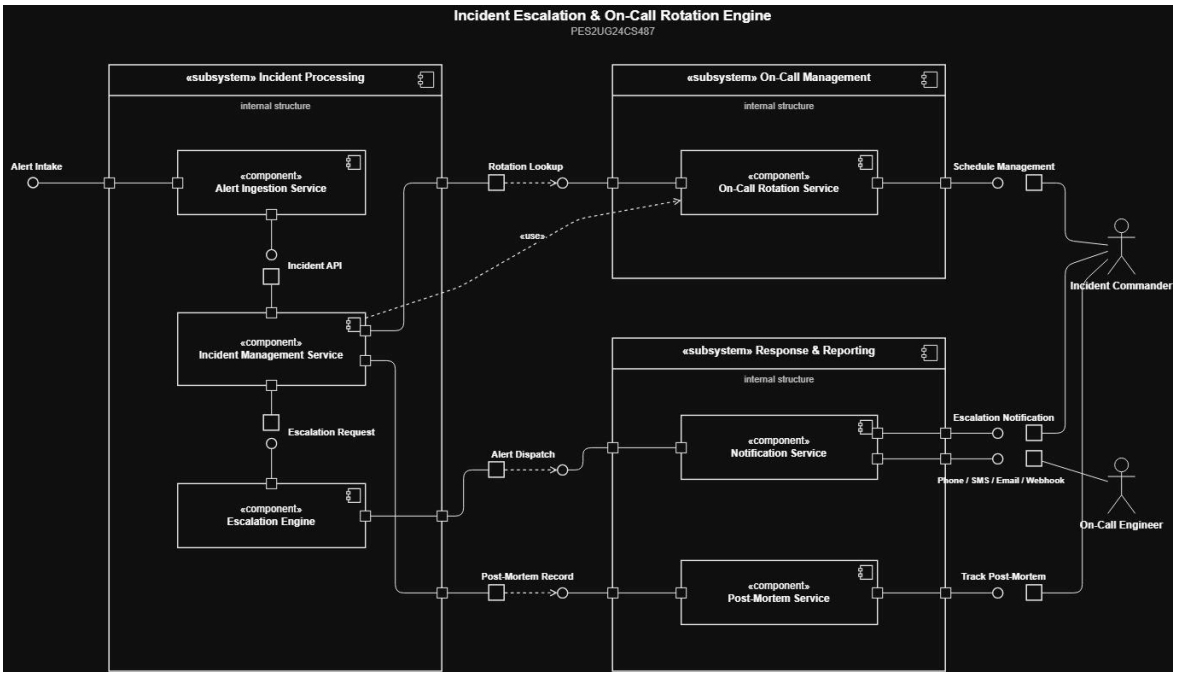
SE LAB -3

Name: Shreya Shenoy
SRN: PES2UG24CS487
Class: 5-H
Date: 16-09-2026

*Component Diagram (made on [draw.io](draw.io) due to license expiry on staruml)*




*Architecture Choice explanation:*

My question for **Incident Escalation & On-Call Rotation Engine** uses a **Microservices Architecture**. The system is divided into separate services for alert ingestion, incident management, on-call rotation, escalation, notification, and post-mortem handling. These services communicate through clearly defined interfaces, while incident and on-call schedule information are maintained by dedicated data repositories. This structure allows each major incident-response function to remain logically independent.

Why I havent chosen Layered or Client-server is because mainly Client-Server is fundamentally just a deployment topology (the engine itself is the server side ) but it doesn't address the multiple operations being performed , while a traditional Layered architecture creates severe bottlenecks and operational coupling for systems with this profile.

*Reason 1: Separation of Incident-Response Functions*

The system performs several distinct operations: receiving monitoring alerts, identifying the active on-call engineer, notifying the responder, detecting acknowledgement timeouts, escalating incidents through the configured hierarchy, and recording post-mortem information. Separating these responsibilities into individual services makes the workflow easier to maintain and modify. For example, changes to the escalation logic can be made within the Escalation Service without requiring changes to alert ingestion or post-mortem handling.

*Reason 2: Fault Isolation and Independent Scaling*

Incident volume may increase during periods of system failure, resulting in many alerts and escalation requests being generated simultaneously. Microservices allow heavily used components such as the Alert Ingestion, Escalation, and Notification Services to be scaled independently. They also provide fault isolation, so a failure in one service does not necessarily prevent other incident-management functions from continuing to operate.

*Security Advantage*

Each service can enforce authentication and authorization at its interface and follow the principle of least privilege when accessing system data. For eg, the Notification Service only needs the information required to deliver an alert and does not need unrestricted access to incident or on-call schedule records. Separating service responsibilities therefore helps limit unnecessary access to sensitive incident information

*Performance Benefit*

Microservices allow individual components to be scaled according to workload. During a large number of simultaneous alerts, the Alert Ingestion, Escalation, and Notification Services can process their respective workloads independently instead of making the entire application scale as one unit. This supports the requirement for rapid alert dispatch, which specifies that notification initiation should occur within 3 seconds of alert ingestion